Introduction to Arrays

An **array** is a collection of elements of the **same data type stored in consecutive memory locations**.

Instead of declaring many variables, we can store multiple values in **one array**.

Without Arrays

int mark1 = 70;

int mark2 = 65;

int mark3 = 80;

int mark4 = 55;

int mark5 = 90;

This becomes difficult to manage.

With Arrays

int marks[5] = {70, 65, 80, 55, 90};

Now all marks are stored in **one variable called marks**.

2. Declaring an Array

Syntax

data_type array_name[size];

Example

int numbers[10];

This creates an array that can store **10 integers**.

3. Accessing Array Elements

Each element in an array has an **index**.

Indexing starts from **0**.

Index	Value
0	10
1	20
2	30
3	40

Example:

```
int numbers[4] = {10,20,30,40};


printf("%d", numbers[0]);
```

Output:

10

4. Using Loops with Arrays

Loops are commonly used to **process array elements**.

Example:

```
#include <stdio.h>

int main()
{
   int numbers[5] = {10,20,30,40,50};

   for(int i = 0; i < 5; i++)
   {
      printf("%d\n", numbers[i]);
   }

   return 0;
}
```

Output

10

20

30

40

50

5. Reading Array Values from the User

Example:

```
#include <stdio.h>

int main()
{
   int marks[5];

   for(int i = 0; i < 5; i++)
   {
      printf("Enter mark %d: ", i+1);
      scanf("%d", &marks[i]);
   }

   for(int i = 0; i < 5; i++)
   {
      printf("%d\n", marks[i]);
   }

   return 0;
}
```

6. Types of Arrays

1. One-Dimensional Array

A simple list of elements.

Example:

int numbers[5];

2. Two-Dimensional Array (Matrix)

Used for **tables or grids**.

Example:

int matrix[3][3];

Example representation:

1 2 3

4 5 6

7 8 9